

От монолита к микросервису

Microservice Architecture

REC Проверить, идет ли запись

Меня хорошо видно & слышно?





Вадим Перепелкин

- Более 15 лет в ИТ
- Занимал разные позиции (developer, team lead, architect)

@VPerepelkin

vup@rambler.ru

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе
#OTUS Microservice-2025-06



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом

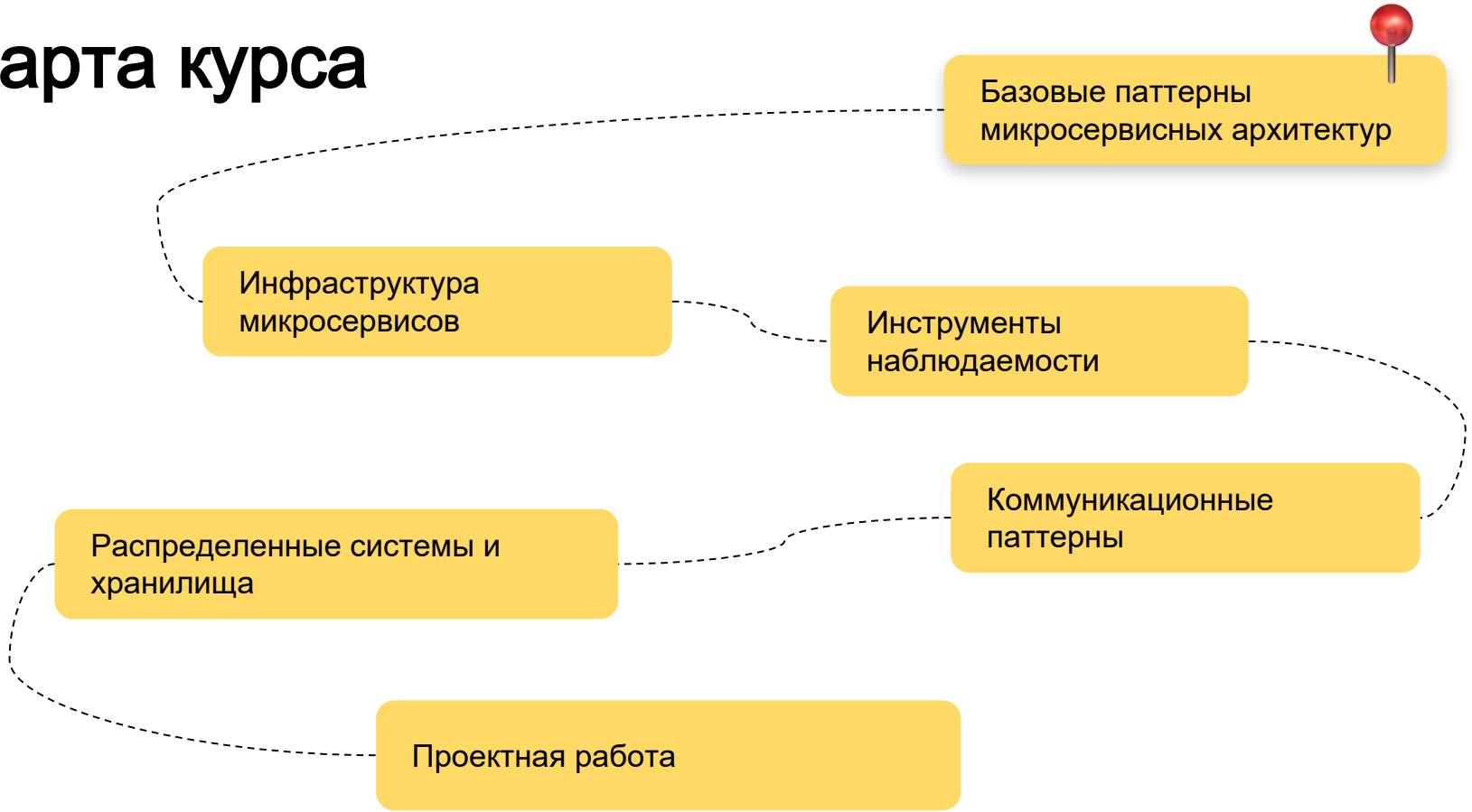


Документ



Ответьте себе или
задайте вопрос

Карта курса


Базовые паттерны микросервисных архитектур
Инфраструктура микросервисов
Инструменты наблюдаемости
Коммуникационные паттерны
Распределенные системы и хранилища
Проектная работа

Маршрут вебинара

- Паттерны/подходы по переходу от монолита в микросервисы
- Паттерн Strangler

Цели вебинара

К концу занятия вы сможете:

1. Применять паттерн Strangler
2. Применять паттерны работы с данными при рефакторинге

Монолит
в микросервисы

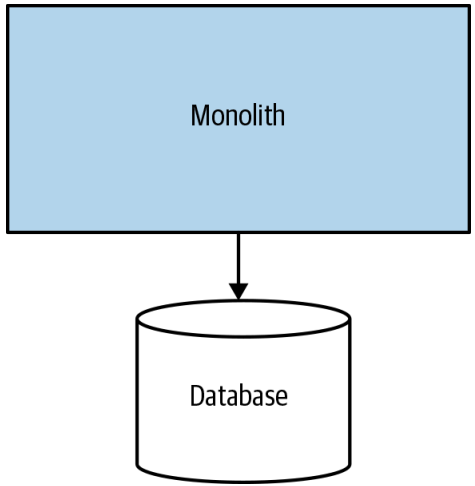
Монолит в микросервисы



У нас есть монолит, как его распилить на микросервисы?

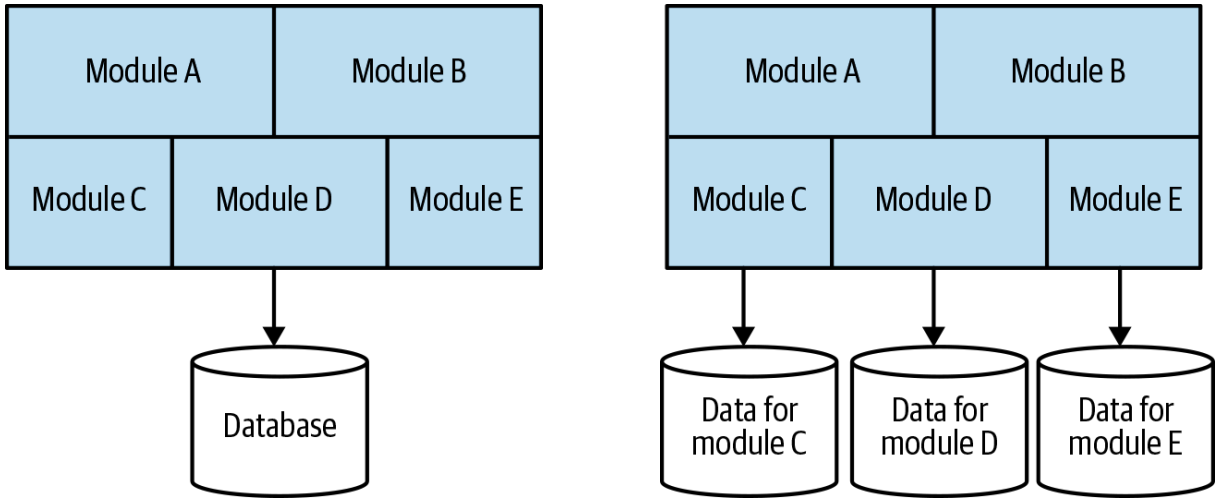
Единый монолит

- **Единый монолит** – монолит, который деплоится как единое целое, содержит в себе всю функциональность



Модульный монолит

- **Модульный монолит** – монолит, который деплоится как единое целое, содержит в себе всю функциональность, но у которого есть возможность деплоить только часть функциональности (некоторый набор модулей).



Зачем?

Прежде чем распиливать «монолит» в микросервисы нужно четкое понимание того, какие плюсы принесет микросервисная архитектура и сколько нужно будет потратить человеко - месяцев и лет для того, чтобы перейти на новую архитектуру.

Если сделан осознанный выбор в сторону микросервисов, то нужно приготовится к долгому и планомерному переходу.



Напишите в чат

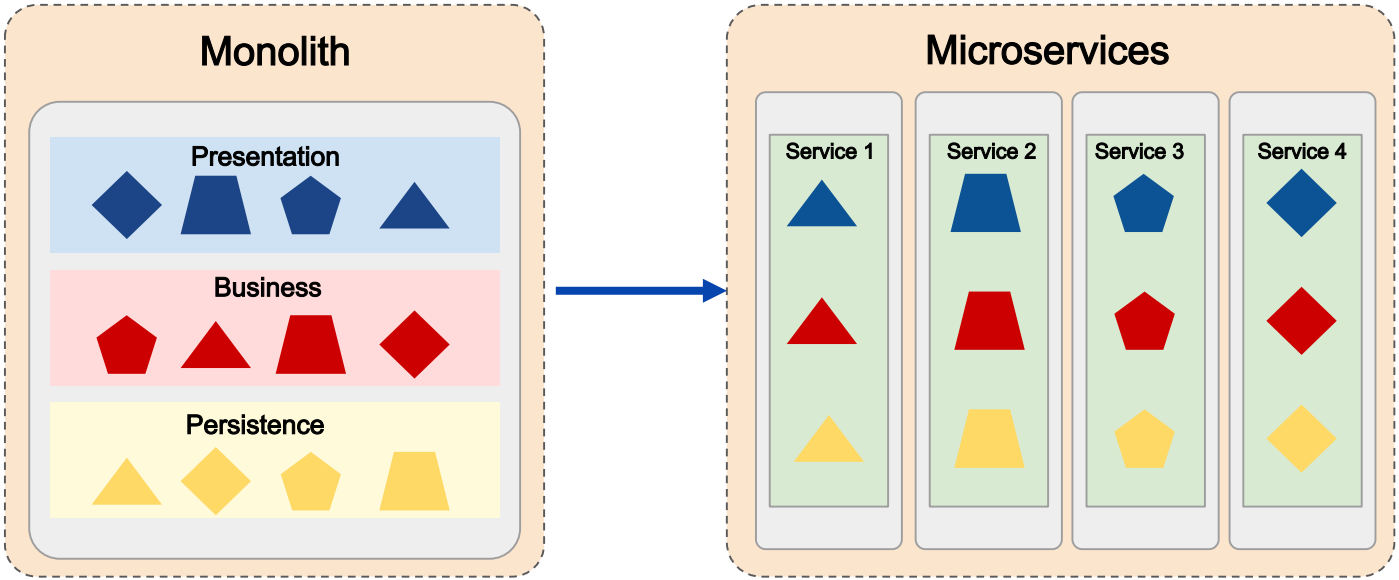
Зачем?

Прежде чем распиливать «монолит» в микросервисы нужно четкое понимание того, какие плюсы принесет микросервисная архитектура и сколько нужно будет потратить человеко - месяцев и лет, для того, чтобы перейти на новую архитектуру.

Если сделан осознанный выбор в сторону микросервисов, то нужно приготовится к долгому и планомерному переходу.

- **Медленная доставка** – сложно разобраться, сложно менять, сложно и долго тестировать
- **Обновления с ошибками** – из-за плохой тестируемости она недостаточна, из-за сложности понимания и изменений много ошибок
- **Плохая масштабируемость** – модули с разными требованиями к ресурсам

Цель



С чего начать? Напишите в чат

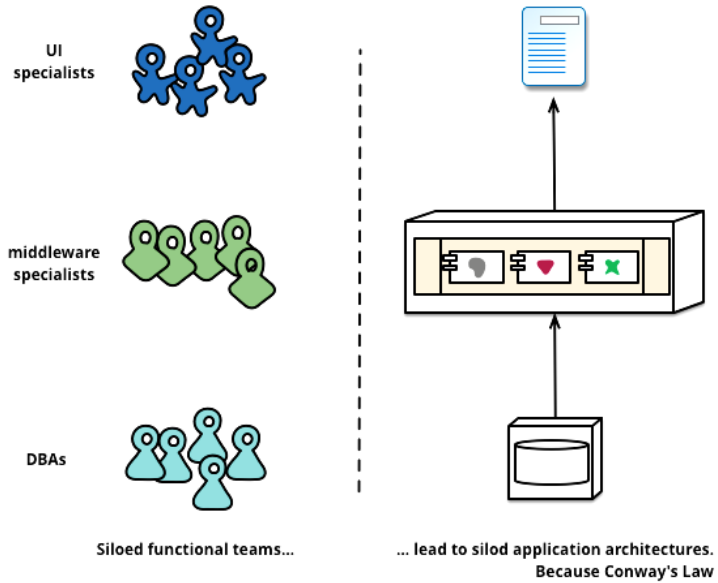
Закон Конвея

Организации, проектирующие системы , ограничены дизайном, который копирует структуру коммуникации в этой организации

© Мелвин Конвей, американский ученый -программист

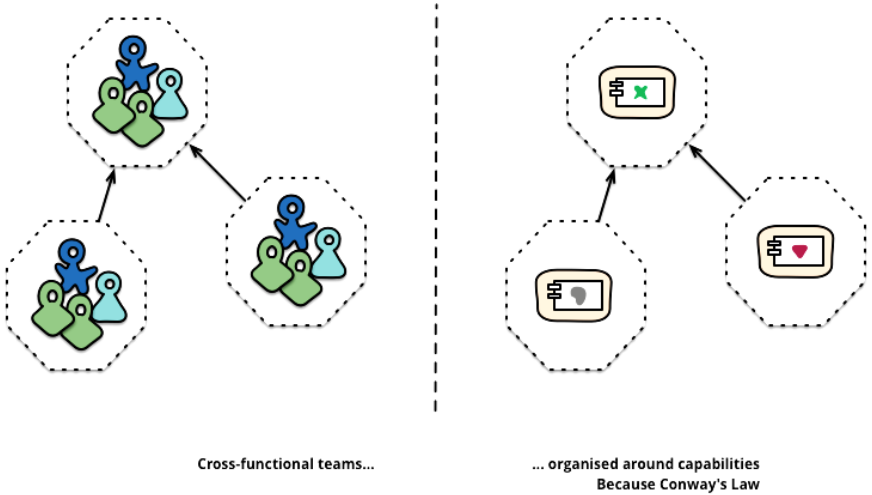
Организационная структура при монолитах

В случае монолитов мы скорее всего будем иметь несколько «специализированных» по функциям команд – DBA, Backend, Frontend и т.д.



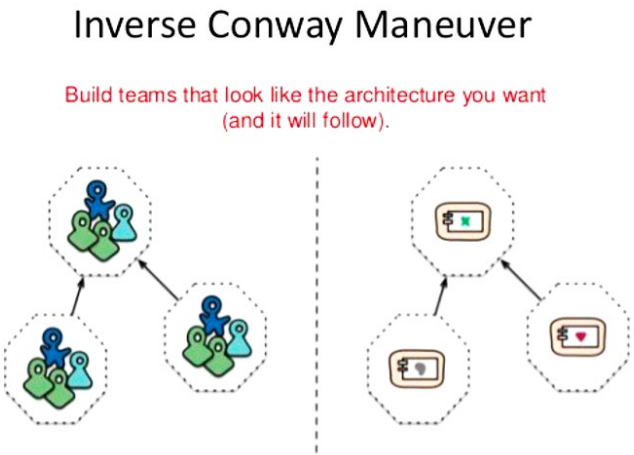
Организационная структура при микросервисах

Микросервисы – это чаще всего самостоятельно деплоятся и границы микросервисов проходят по business capabilities (продукту). Поэтому чаще всего для микросервисной архитектуры характерны кросс-функциональные продуктовые команды.



Начните с орг. дизайна

Измените организационную структуру и архитектура сама постепенно станет повторять организационный дизайн.



Подходы

- Если вы в яме – перестаньте копать
- Переписать все
- Переписывать постепенно

Стоимость изменений

Как и любое архитектурное решение, переход от монолита к микросервисам должен происходить постепенно. Решения должны быть как можно менее рискованным (возвратными). При этом сам переход займет больше времени, но будет более предсказуемым и вероятность успеха будет больше

Смена провайдера
Смена публичного API

Внедрение нового языка программирования в организации
Смена базы данных

Эксперименты с новым языком программирования
Выбор open source библиотеки



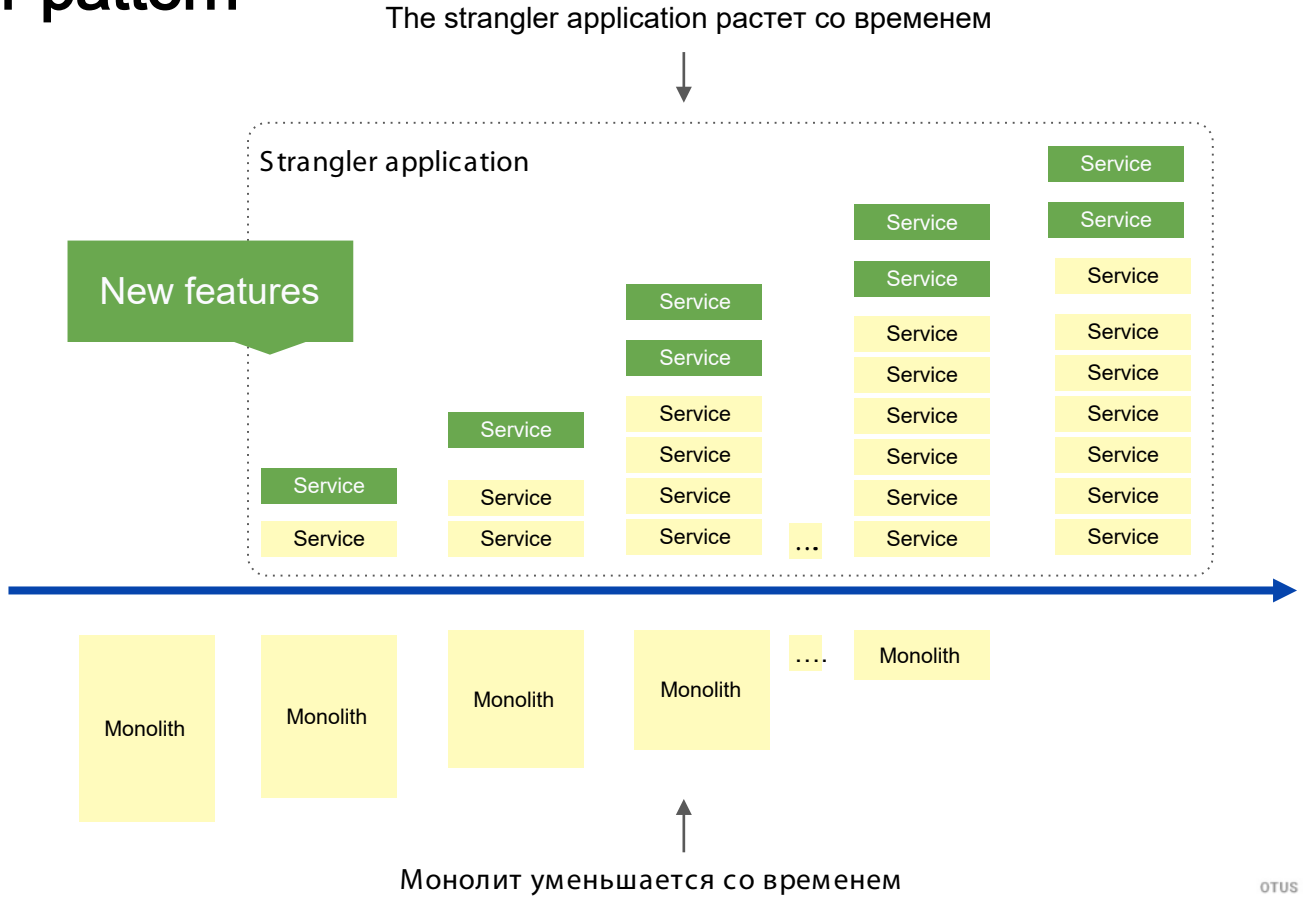
Необратимые

Требуется глобальная поддержка
Высокая сложность/невозможность изменить решение
Требуется тщательное продумывание решения

Обратимые

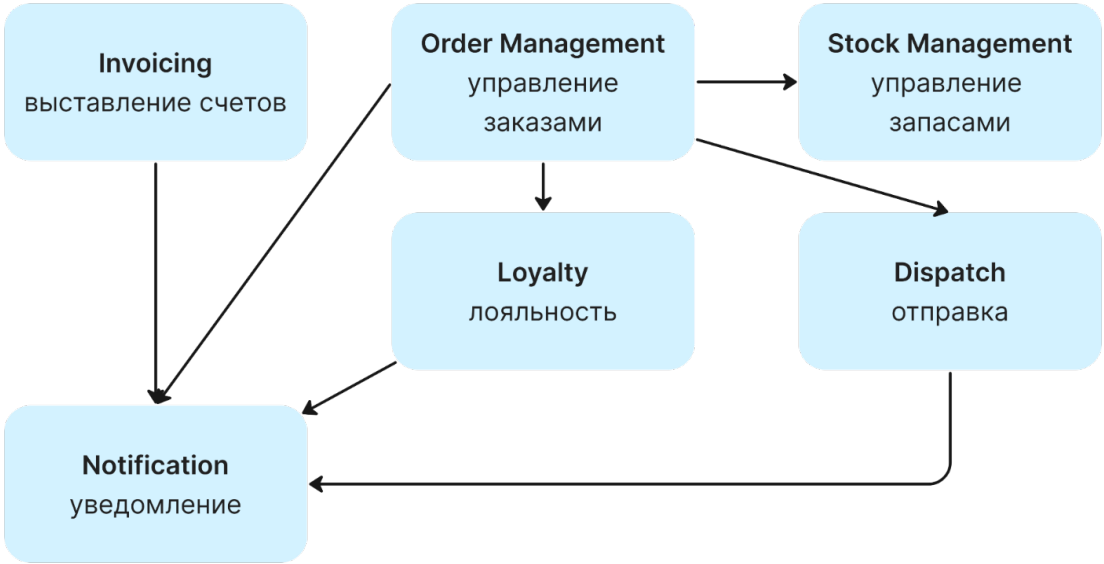
Более локализованное принятие решений
Низкая стоимость отмены решения
Приемлемо принятие решений ad hoc

Strangler pattern



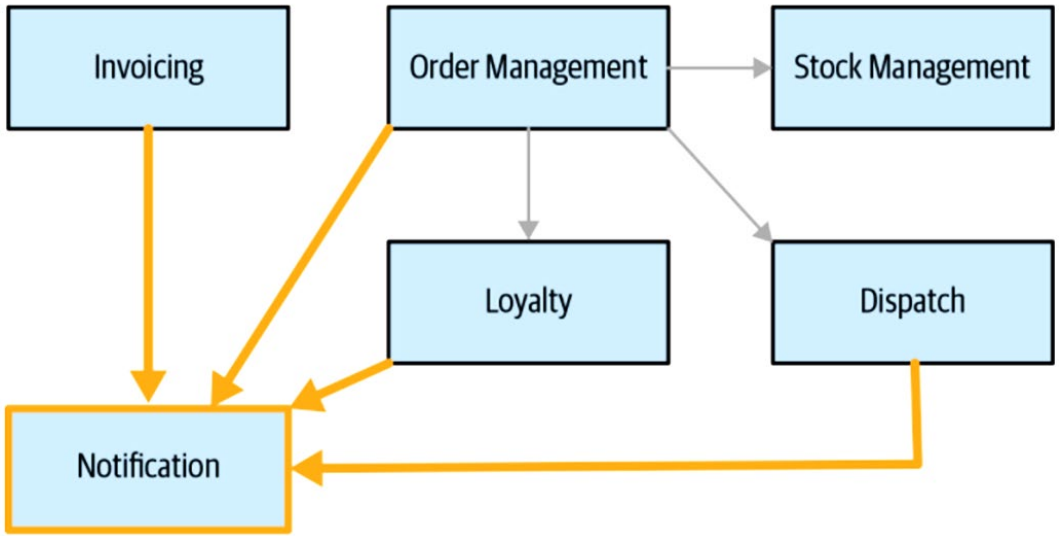
Какие сервисы вытаскивать в первую очередь?

Предположим, что в результате практик декомпозиции сервисов мы приняли решение разделить сервисы следующим образом.



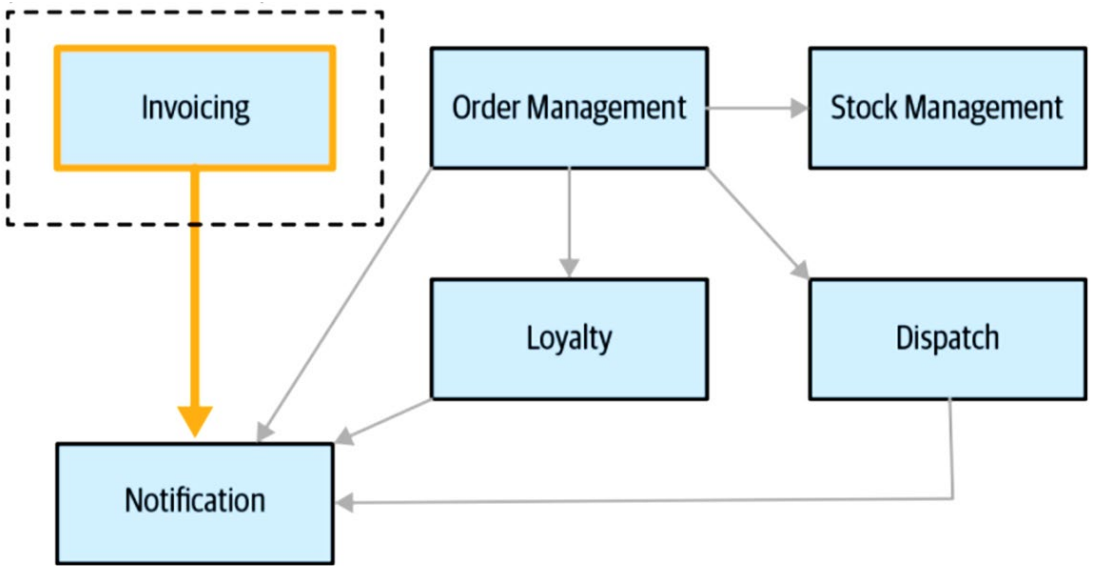
Какие сервисы вытаскивать в первую очередь?

Чем больше зависимостей требует будущий сервис, тем больше придется переписывать и тем сложнее будет миграция.



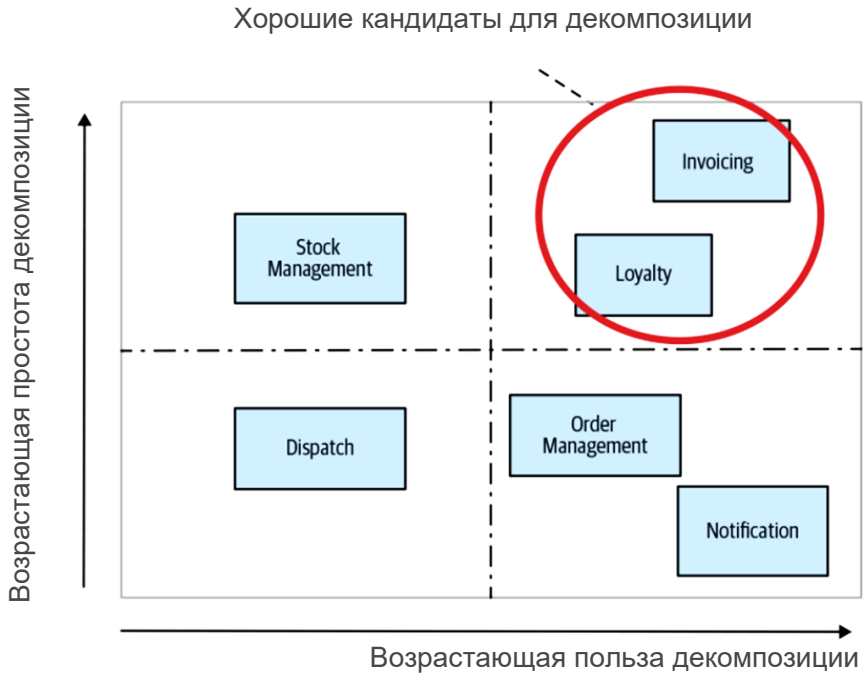
Какие сервисы вытаскивать в первую очередь?

Чем меньше зависимостей требует будущий сервис, тем меньше придется переписывать и тем проще будет миграция.



Какие сервисы вытаскивать в первую очередь?

Лучше вытаскивать те сервисы, которые принесут больше всего профита и которые легче всего вытащить.

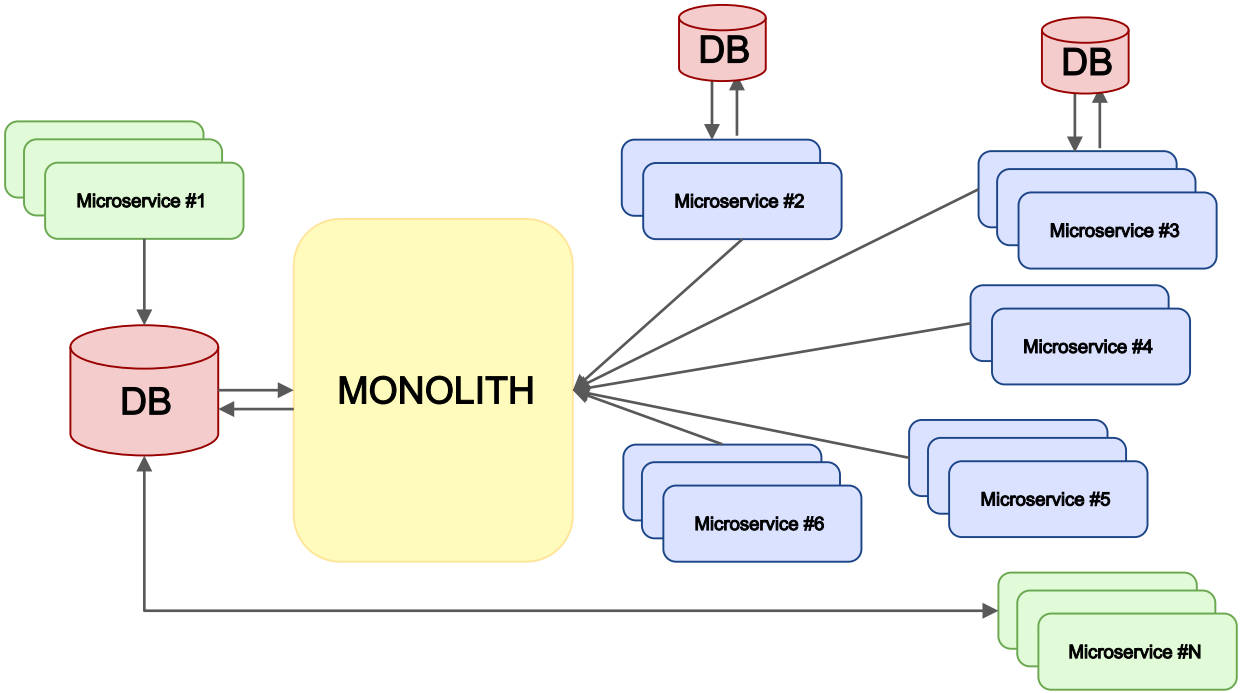


Какие сервисы вытаскивать в первую очередь?

- **Ускорение разработки** . Если согласно плану какая-то часть вашего приложения будет активно развиваться на протяжении следующего года, разработку можно ускорить путем извлечения ее в сервис.
- **Решение проблем с производительностью, масштабируемостью и надежностью** . Если определенная часть вашего приложения ненадежна или имеет проблемы с производительностью или масштабируемостью, будет полезно преобразовать ее в сервис.
- **Возможность извлечь какие -то другие сервисы** . Иногда из-за зависимостей между модулями извлечение одного сервиса упрощает извлечение другого.

Микромонолит

Типичной ошибкой является вытаскивать те сервисы, которые проще всего вытащить, но которые не приносят существенных плюсов. Тогда получается микромонолит – монолит, в котором находится основная бизнес логика + набор (чаще всего инфраструктурных) микросервисов, которые не часто меняются.



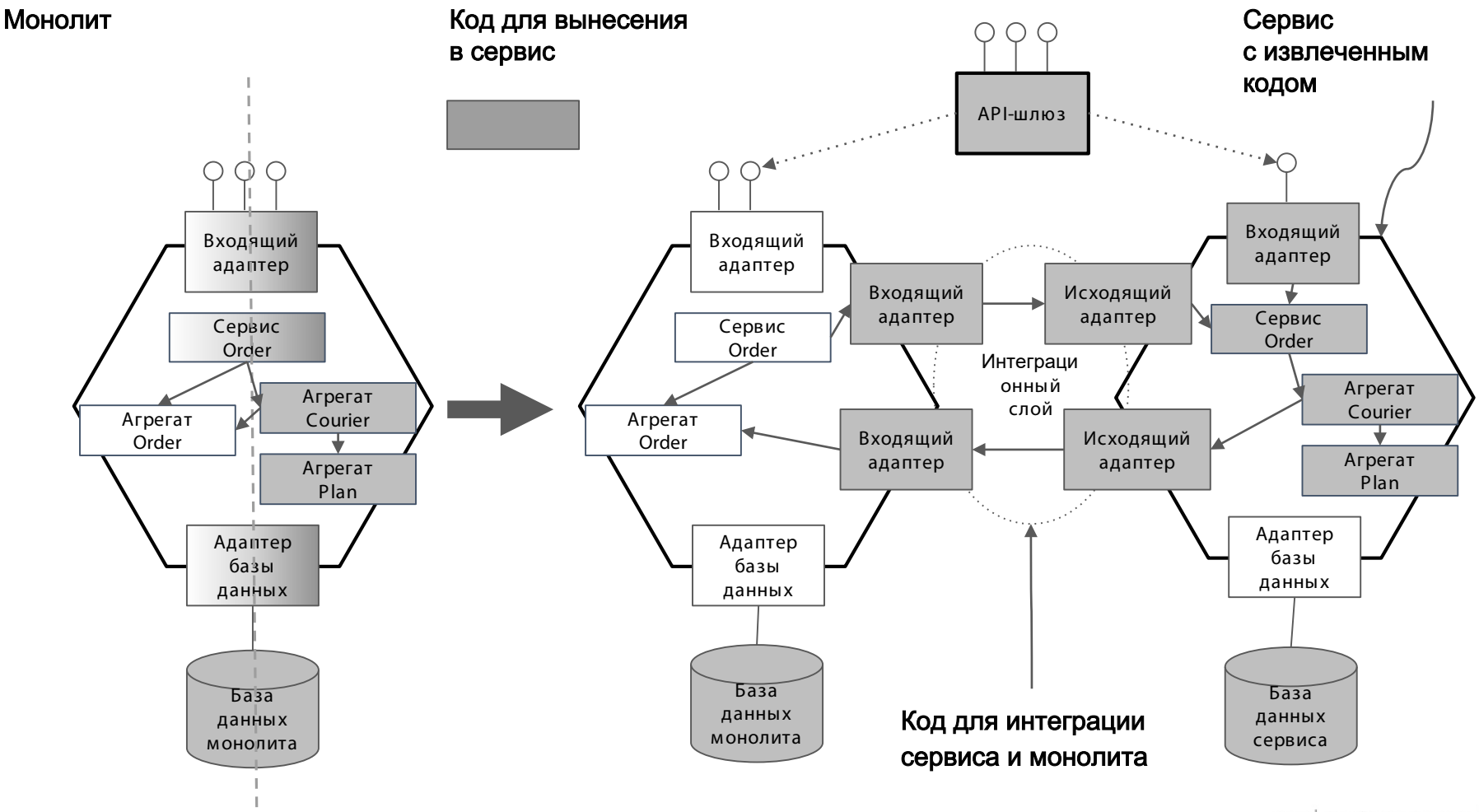
Извлечение функционала в сервис

- Создаем новый микросервис, который повторяет функциональность из монолита
- Переводит внешнием запросы на запросы из монолита в микросервис

Извлечение функционала в сервис - транзакции

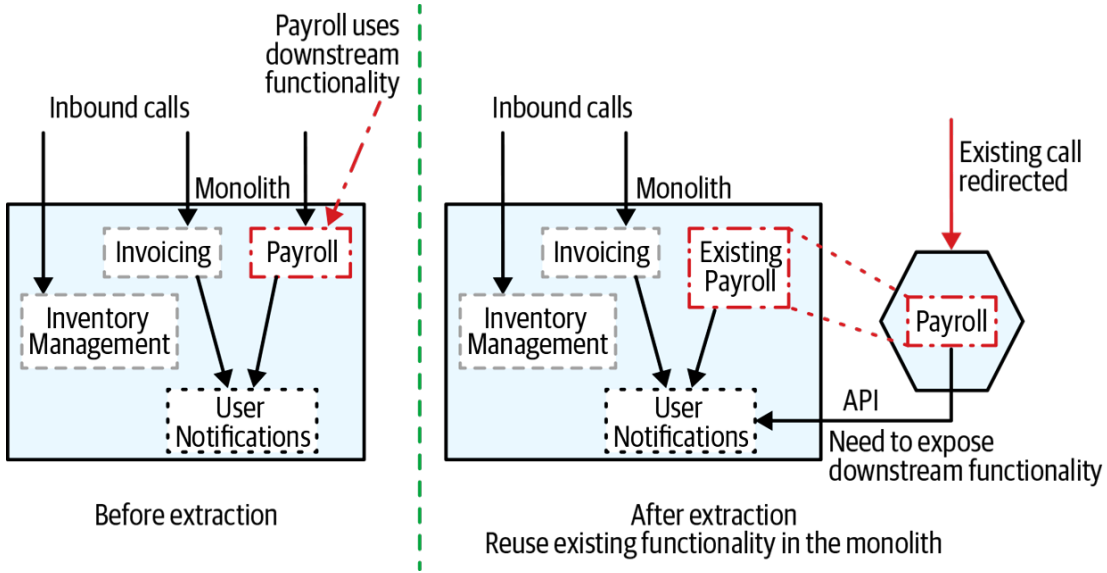
1. Проверить детали заказа
2. Клиент может размещать заказ?
3. Авторизовать карту
4. Создать заказ

1. В монолите
   a. Клиент может размещать заказ?
   b. Создать заказ в состоянии PENDING
2. В сервисе
   a. Проверить детали
   b. Создать заявку в статусе PENDING
3. В монолите
   a. Авторизовать карту
   b. Изменить статус на APPROVED
4. В сервисе - изменить статус на APPROVED


Монолит
Код для вынесения в сервис
Сервис с извлеченным кодом
Входящий адаптер
Сервис Order
Агрегат Order
Агрегат Courier
Агрегат Plan
Адаптер базы данных
База данных монолита
API-шлюз
Входящий адаптер
Сервис Order
Агрегат Order
Входящий адаптер
Исходящий адаптер
Интеграционный слой
Входящий адаптер
Исходящий адаптер
Адаптер базы данных
База данных монолита
Входящий адаптер
Сервис Order
Агрегат Courier
Агрегат Plan
Адаптер базы данных
База данных сервиса
Код для интеграции сервиса и монолита

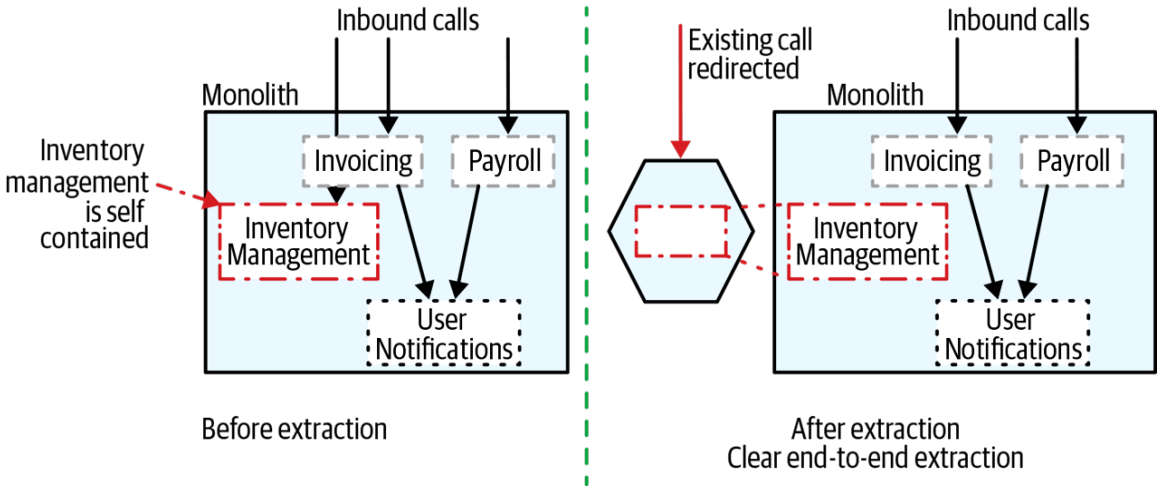
Strangler pattern

Иногда хочется вытащить сервис, который имеет зависимости внутри монолита.
В этом случае придется делать интеграции, которые потом придется рефакторить.



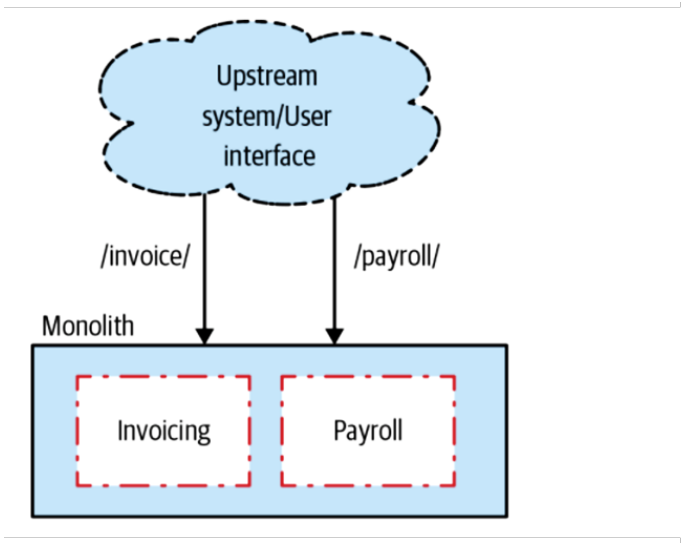
Strangler pattern

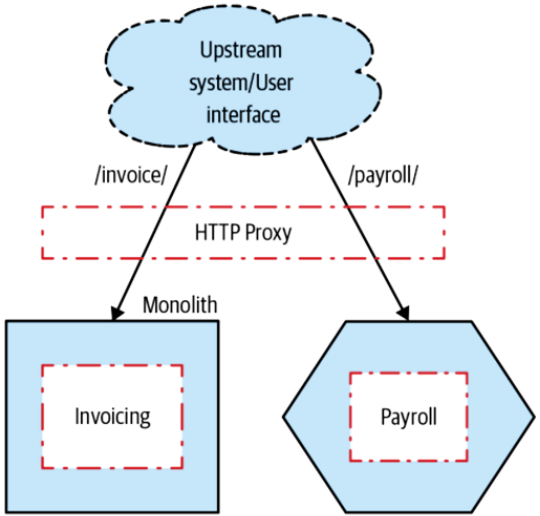
Таким образом можно вытаскивать сервисы, которые имеют не очень много зависимостей и представляют собой «цельный» сервис.



Strangler pattern

Для того, чтобы управлять трафиком, скорее всего придется настраивать балансер (прокси), либо его в явном виде добавлять.



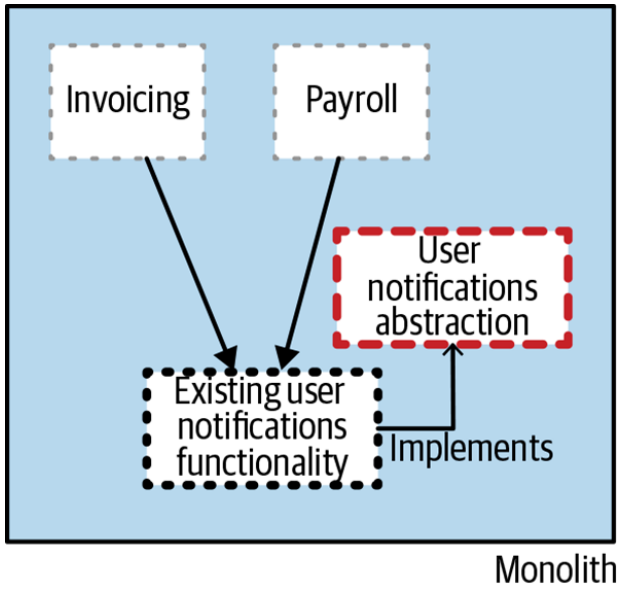


Branch by abstraction

Если мы хотим декомпозировать сервис, который находится «внутри»
и внешний трафик до которого не доходит, то можно воспользоваться другим паттерном – Branch by abstraction

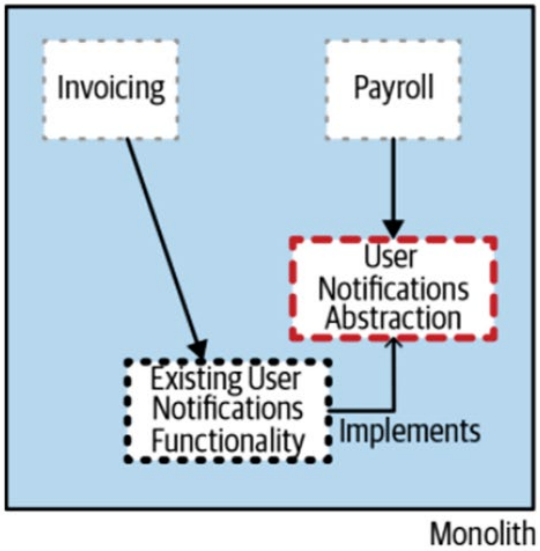
Branch by abstraction

- Выделяем абстракцию (интерфейс)

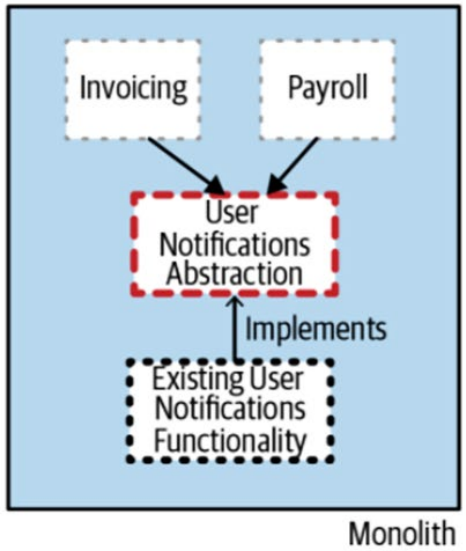


Branch by abstraction

- Используем «абстрактный» интерфейс во всех местах



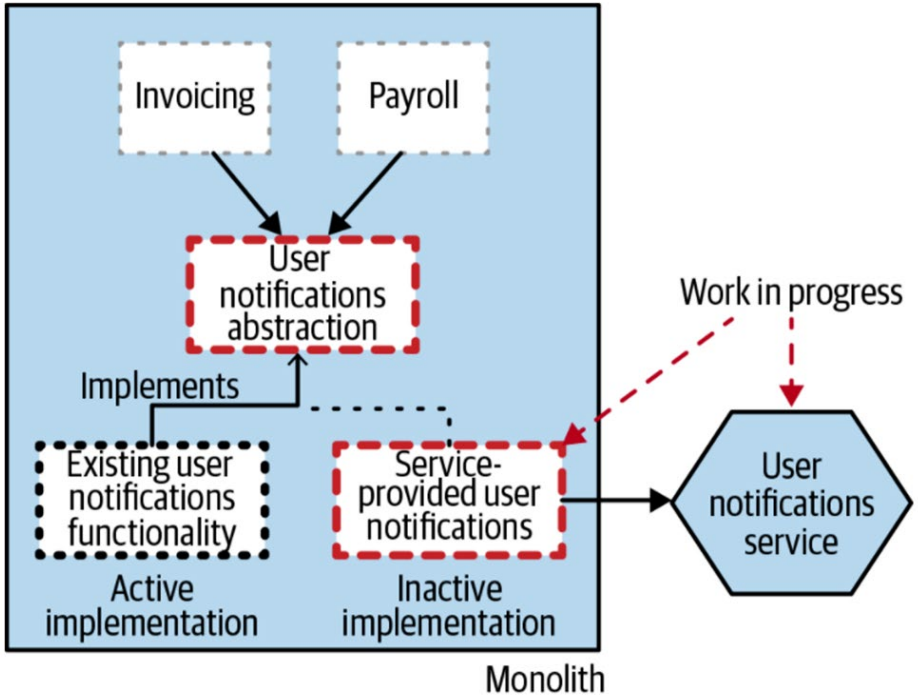
Change Payroll to use the new abstraction



Change Invoicing to use the new abstraction

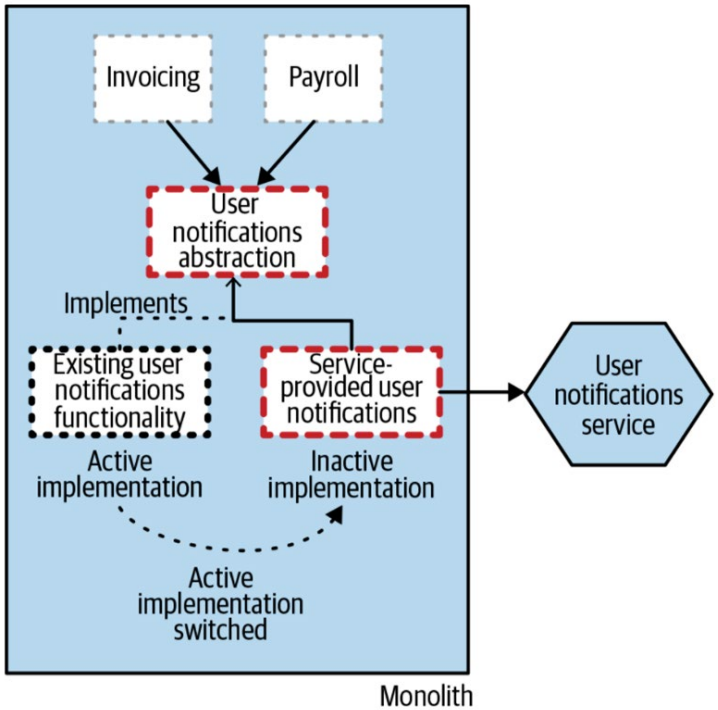
Branch by abstraction

- Создаем сервисную реализацию интерфейса



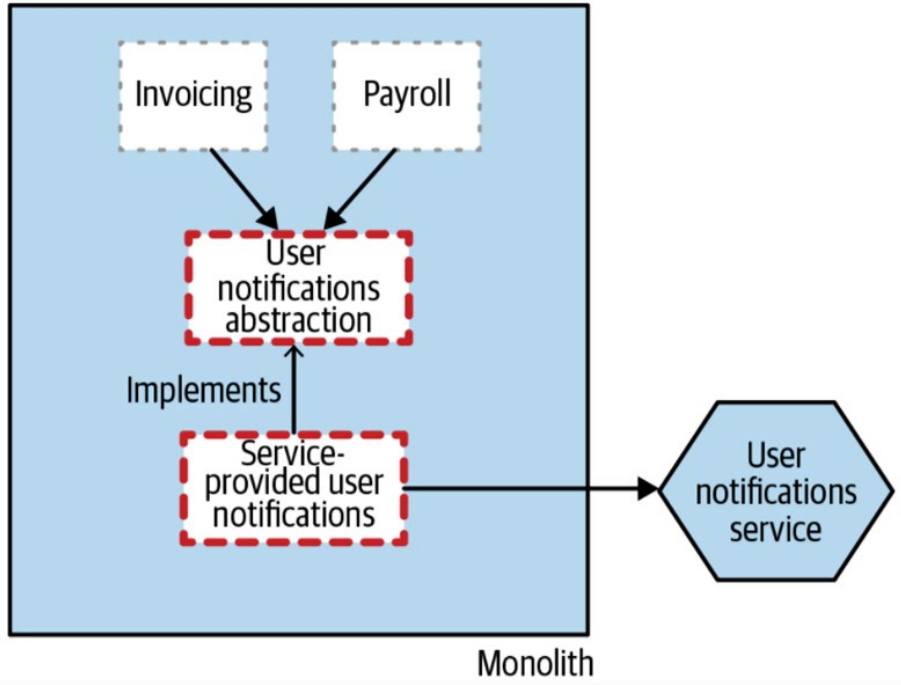
Branch by abstraction

- Переключаем на сервисную реализацию

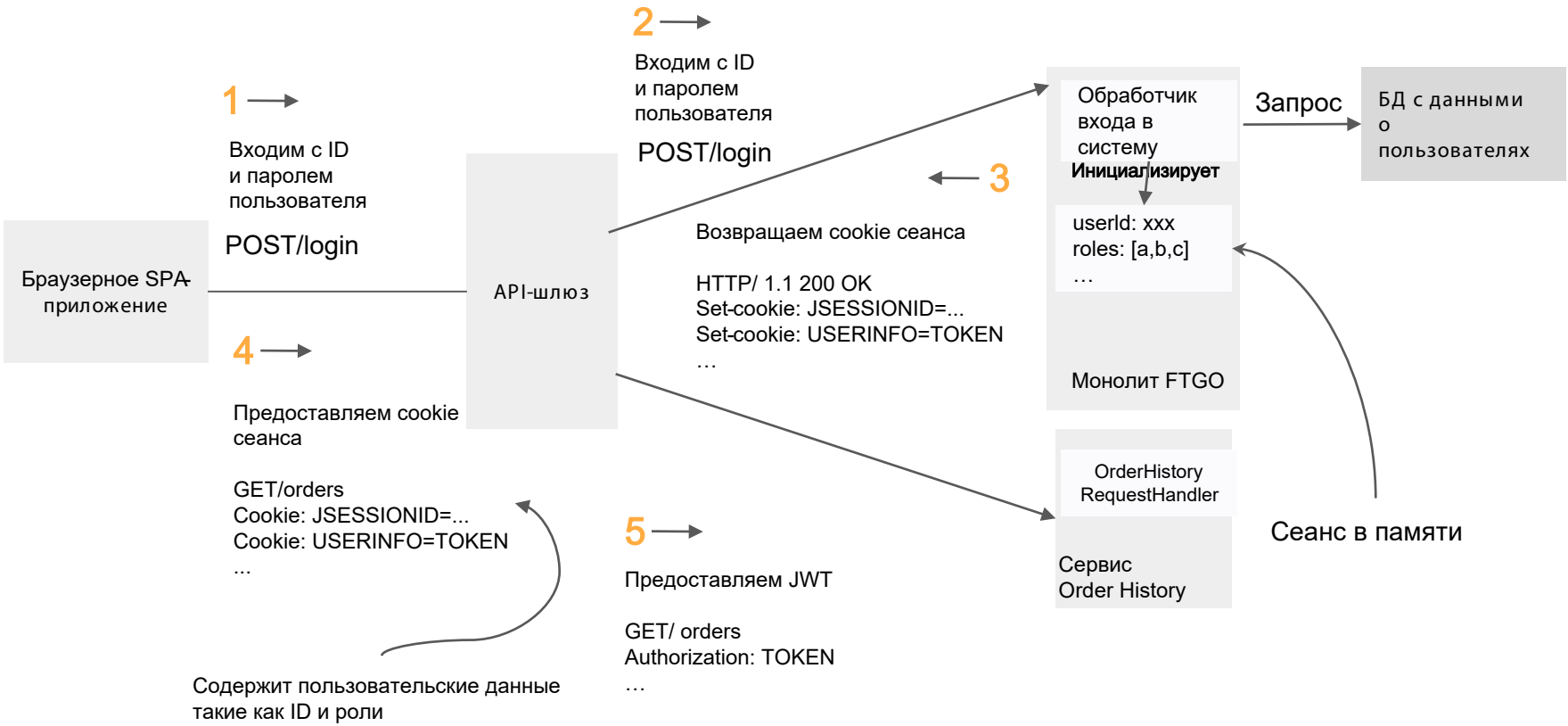


Branch by abstraction

- Очищаем код от старой реализации



Аутентификация и авторизация



Вопросы ?

\+ если есть вопросы

– если вопросов нет

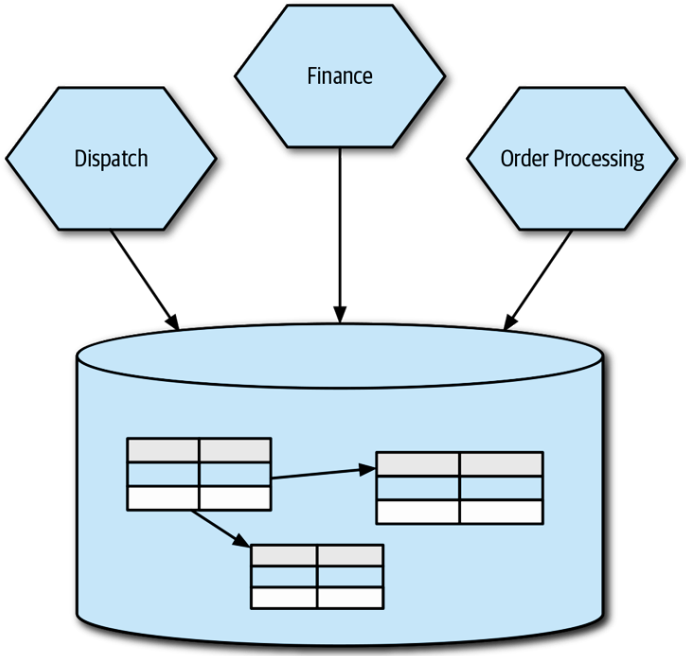
Переход от БД монолита

Декомпозиция БД

- В монолите БД (и схемы) одна. При выделении сервиса мы также должны решить, как мы будем вытаскивать данные из монолита.

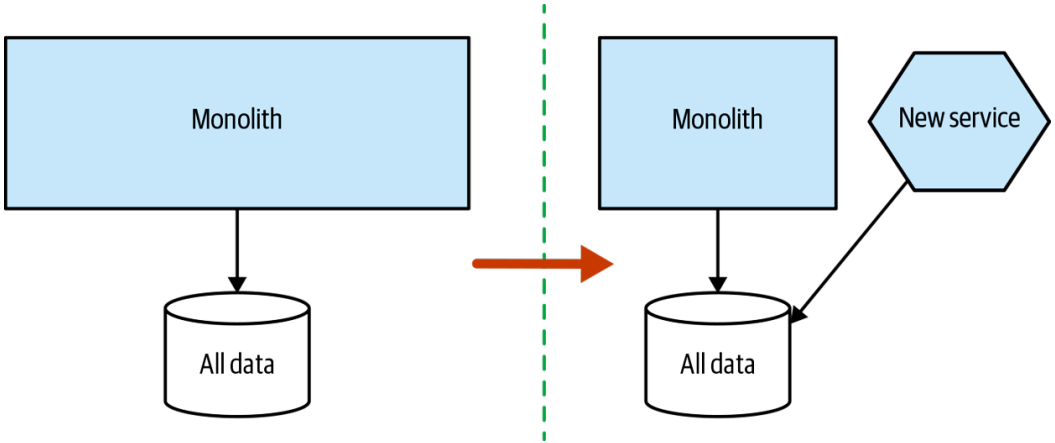
Одна БД для всех сервисов

- В принципе мы можем оставить всё как есть и оставить одну БД для всех сервисов.



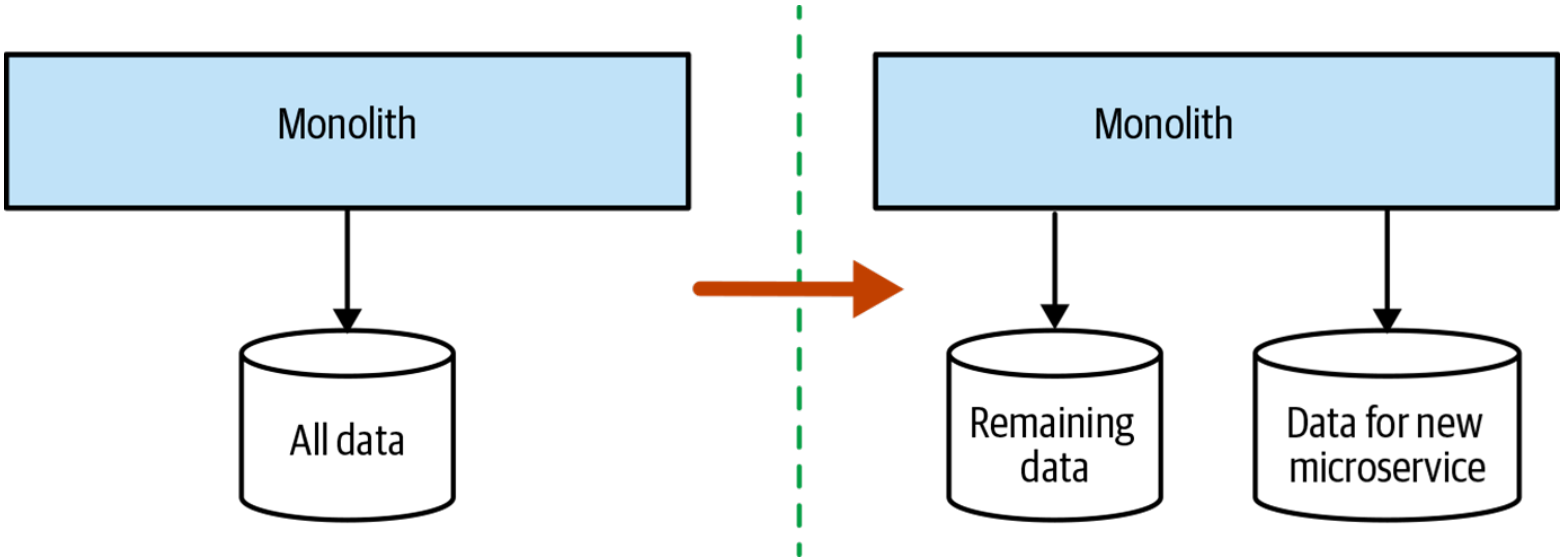
Выделяем сервис, потом данные

- Сначала выделяем сервис, потом отделяем данные.
- Основная проблема в том, что шаг 2 часто не делают и остаются с общей БД.

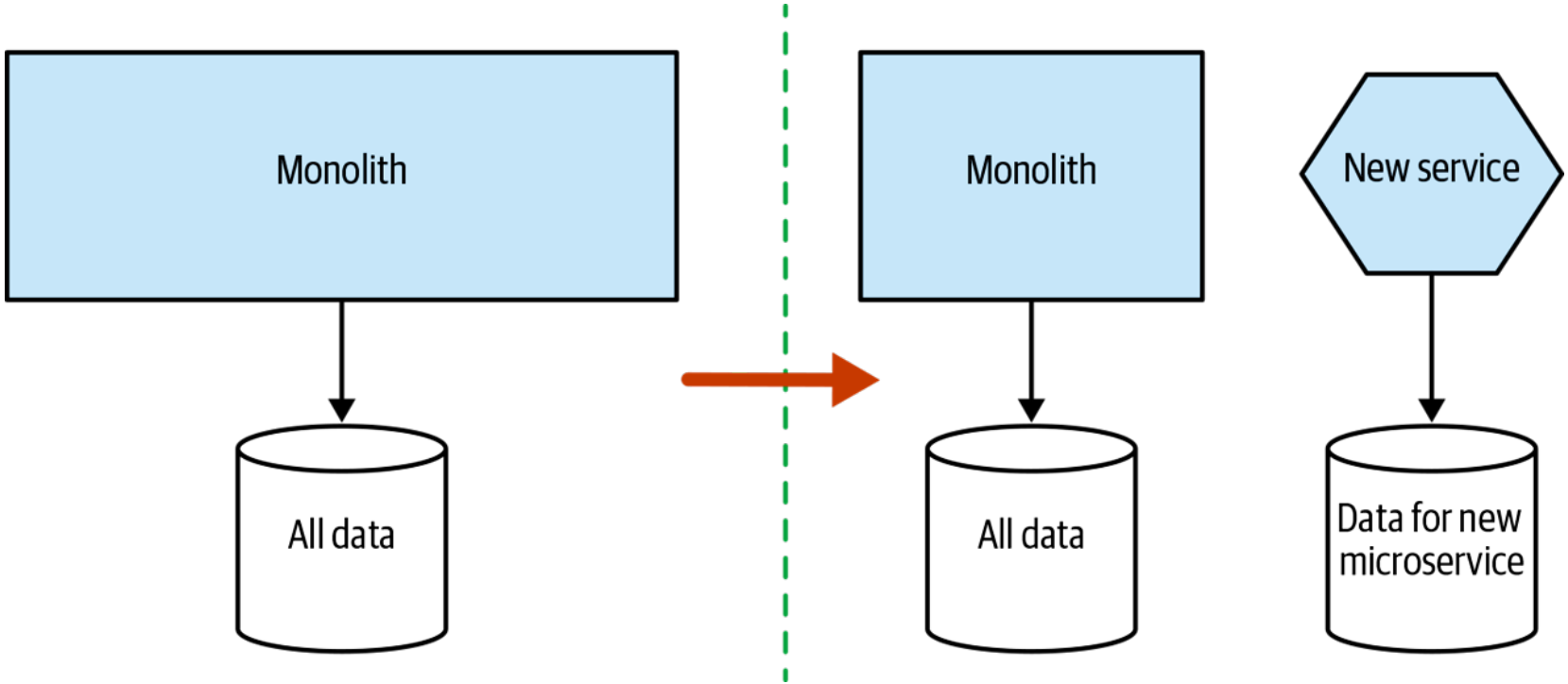


Выделяем сервис, потом данные

- Сначала отделяем данные, потом выделяем сервис.

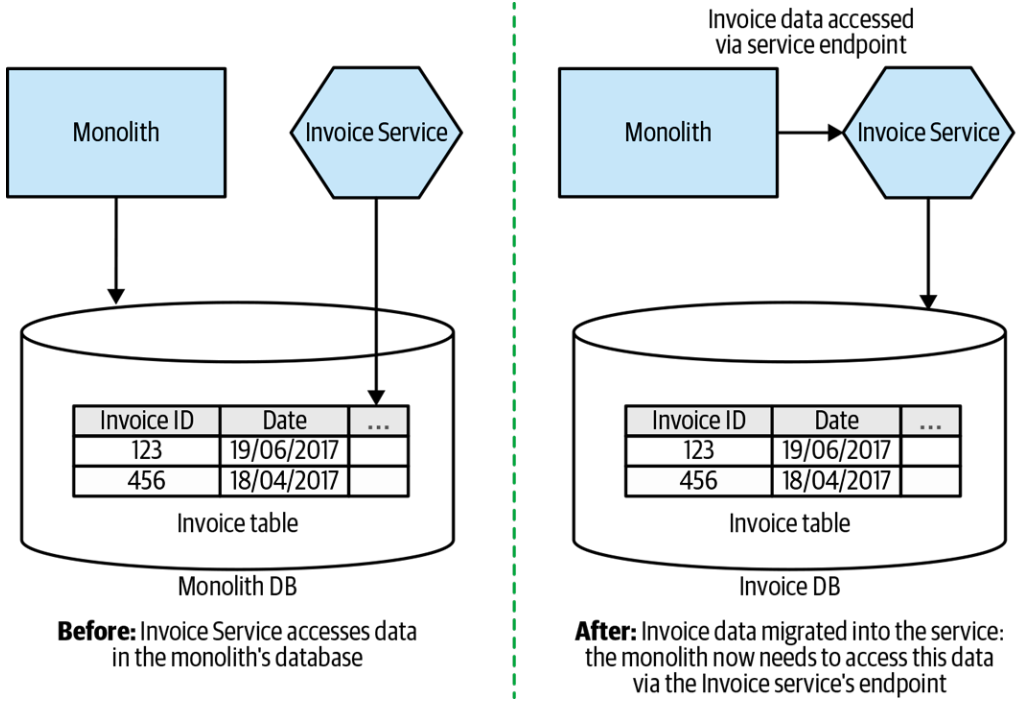


Выделяем данные и сервис одновременно



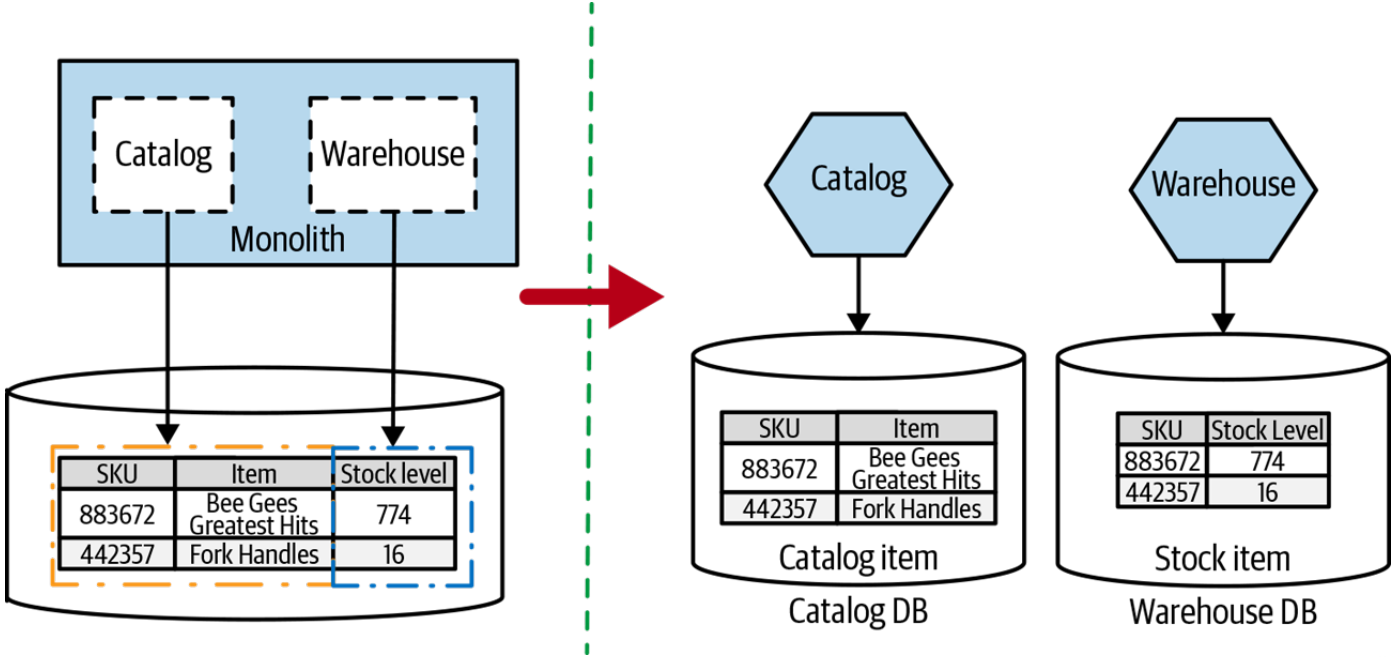
Меняем ответственность данных

- Доступ к данным (даже в рамках одной БД) осуществляется через сервис



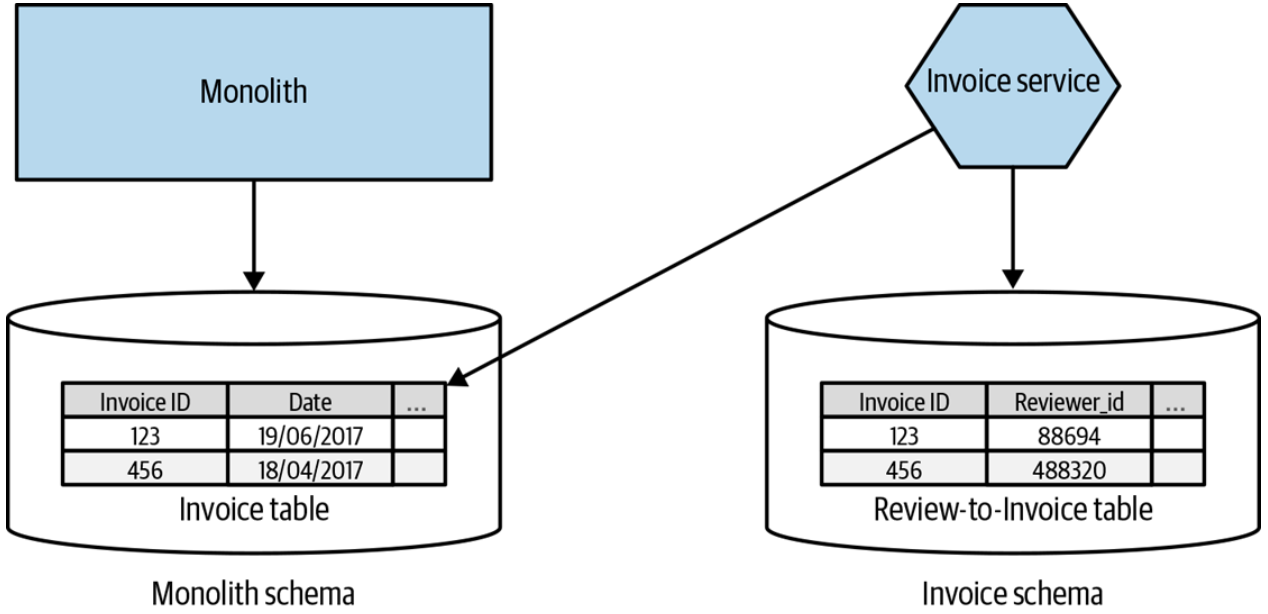
Разделение табличек

- Иногда даже можно начать с разделения табличек в рамках одной БД (схемы), которые потом будут разнесены по разным сервисам



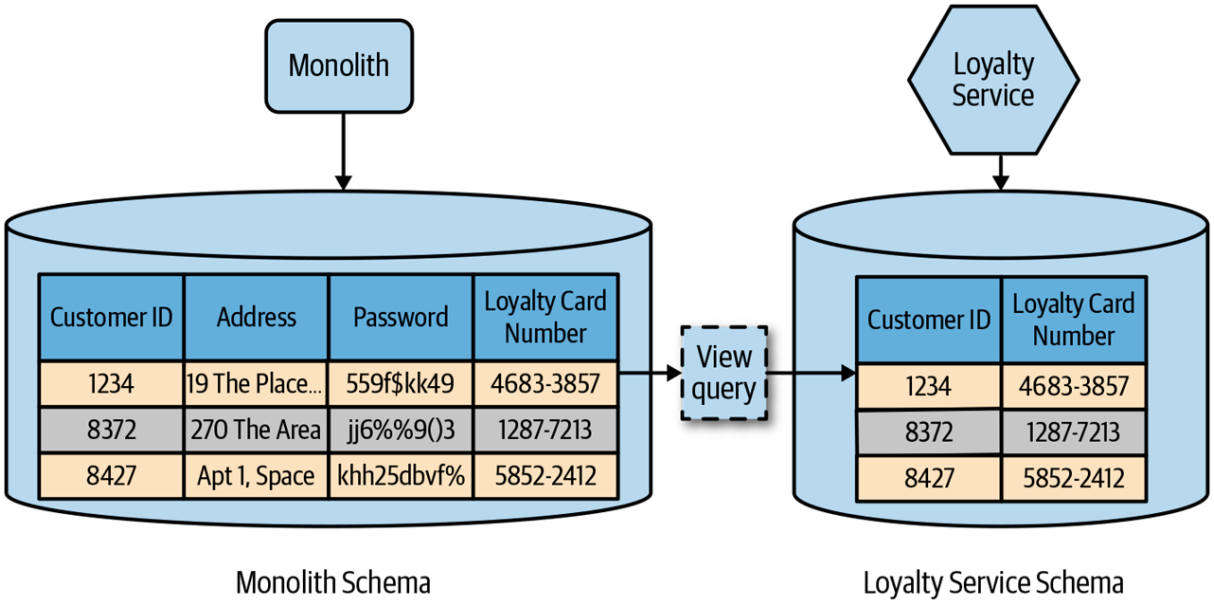
Данные разделенные по схемам

- А иногда можно часть данных унести в схему нового сервиса, а часть оставить в монолите.



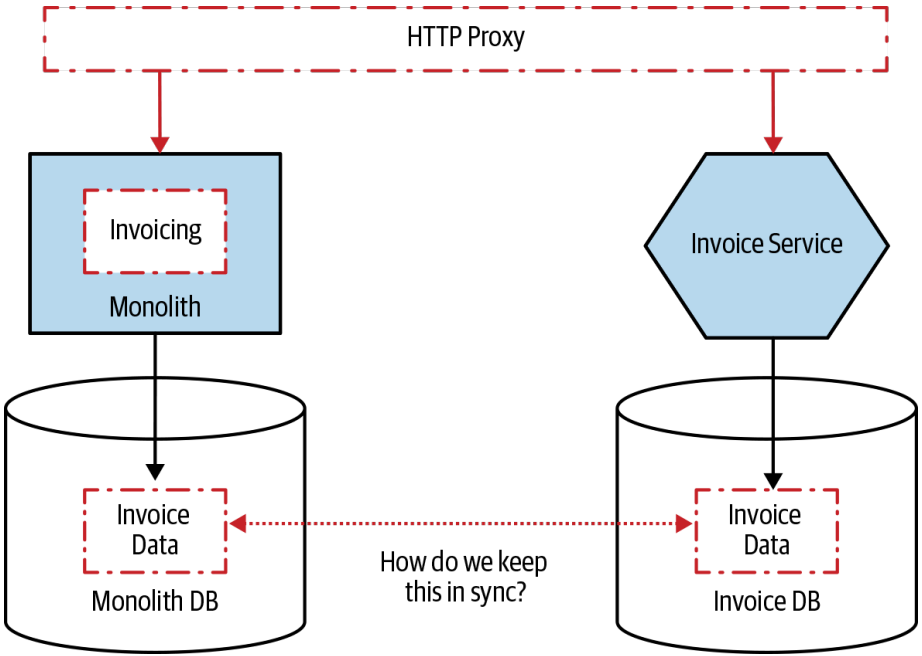
Предоставляем доступ к данным через view

- Можно положить данные не в разные физические БД, а в разные схемы на одном сервере, тогда, можно, например, в монолите сделать view из схемы другого сервиса

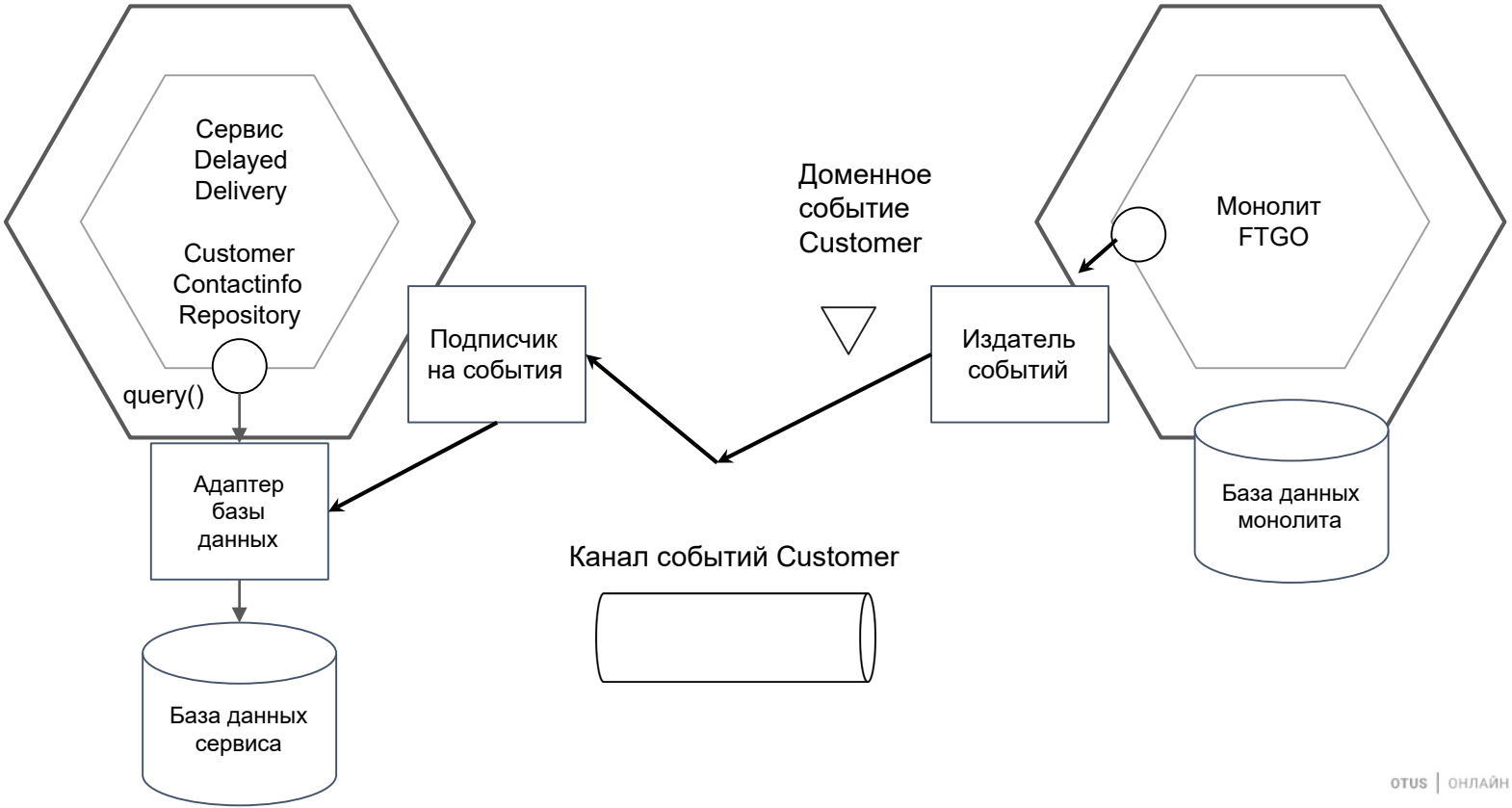


Как обеспечить консистентность данных в разных БД?

Если же все же данные у нас находятся в разных БД, каким образом обеспечить консистентность этих данных, если предположить, что эти данные все еще нужны.



Синхронизация данных - доменные события

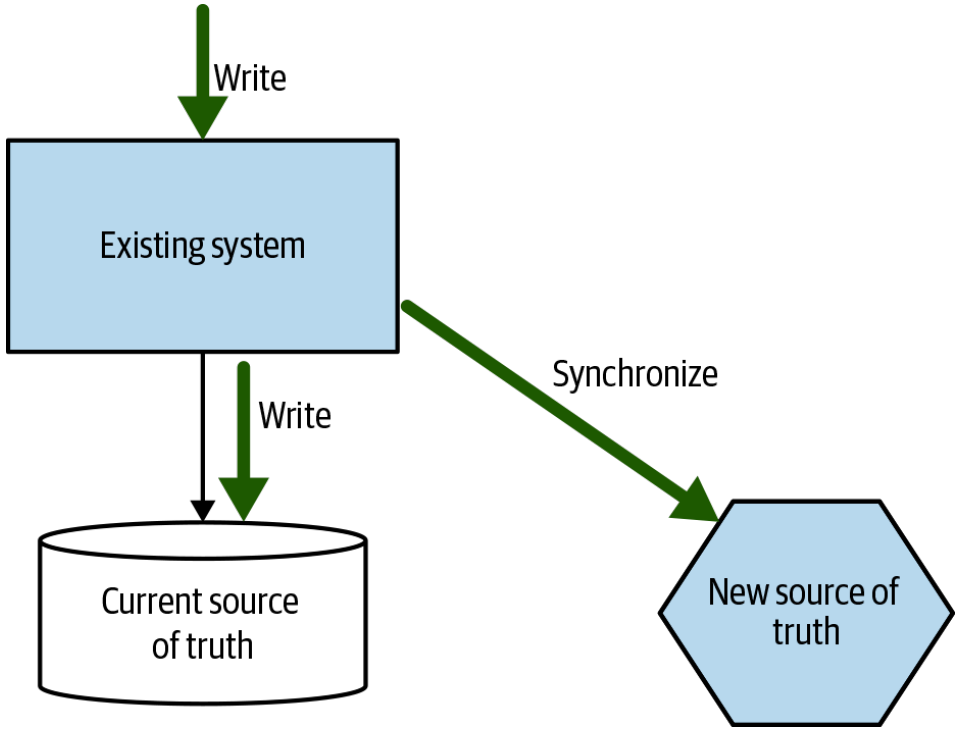


Синхронизация данных из приложения

Текущая система (монолит) должна будет писать сразу в 2 источника (на время миграции) в старый источник данных и в новый.

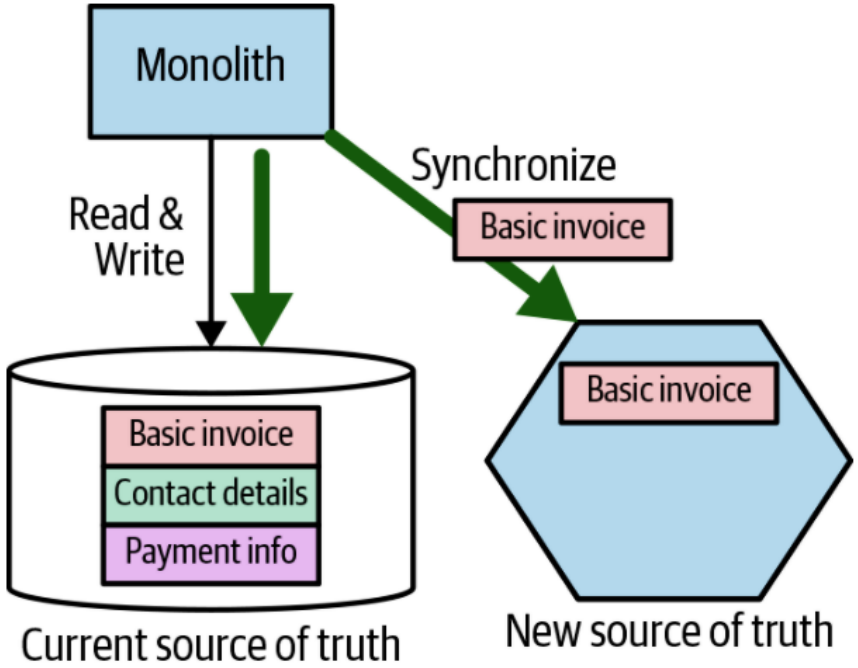
С течением времени потребители должны мигрировать в новый источник.

После окончательной миграции можно старый источник удалять.



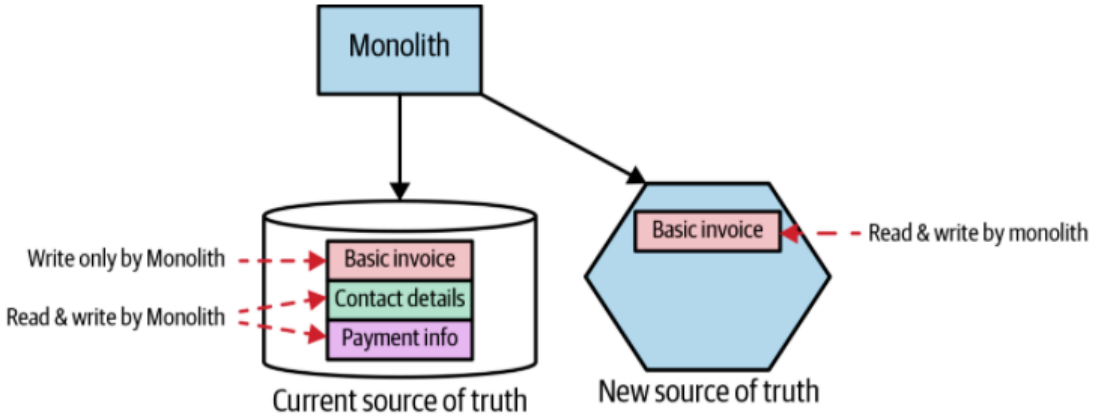
Синхронизация данных из приложения

Можно инкрементально добавлять в новый источник данных данные.



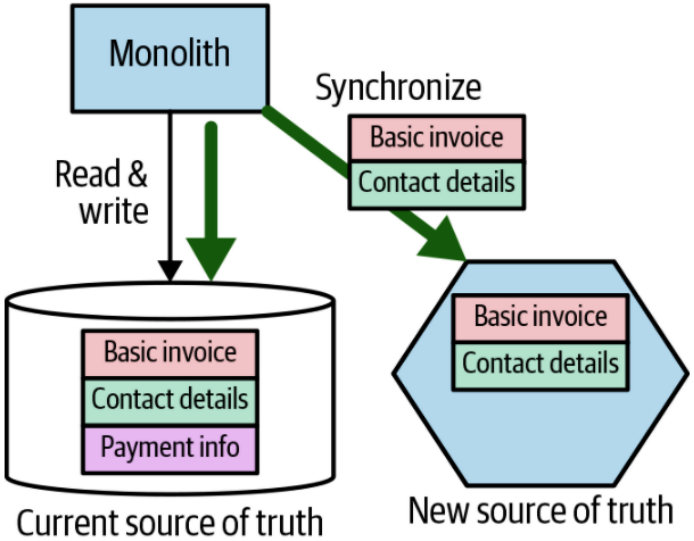
Синхронизация данных из приложения

Можно инкрементально добавлять в новый источник данных данные.



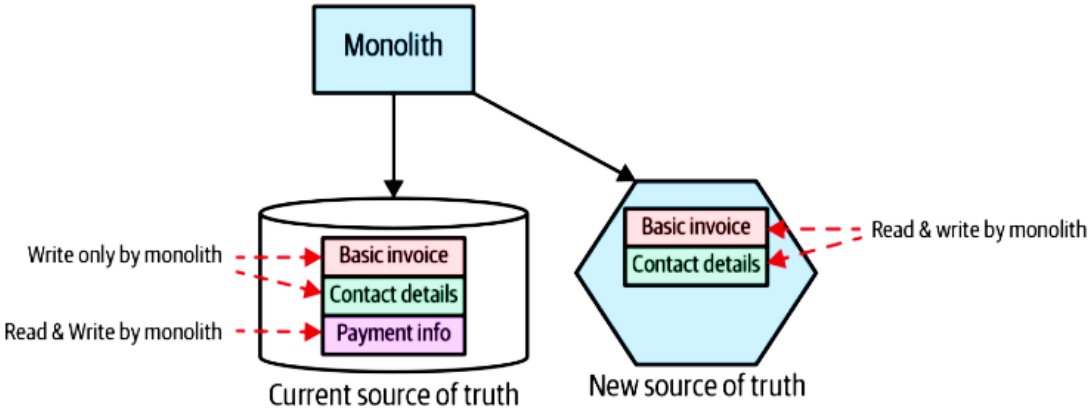
Синхронизация данных из приложения

Можно инкрементально добавлять в новый источник данных данные.



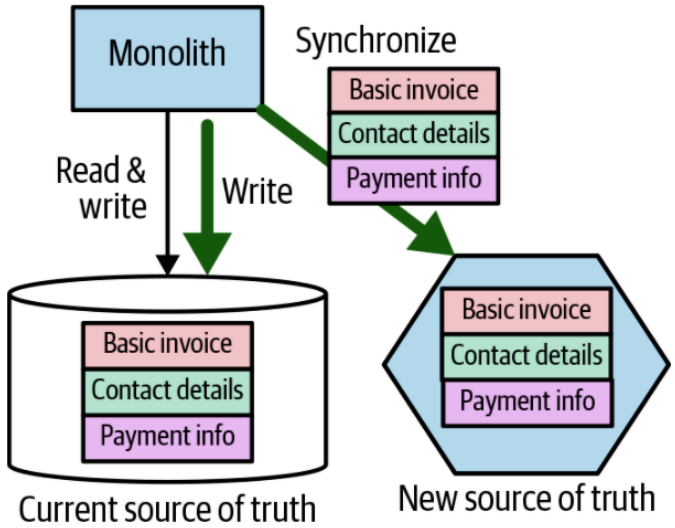
Синхронизация данных из приложения

Можно инкрементально добавлять в новый источник данных данные.



Синхронизация данных из приложения

Можно инкрементально добавлять в новый источник данных данные.



Вопросы ?

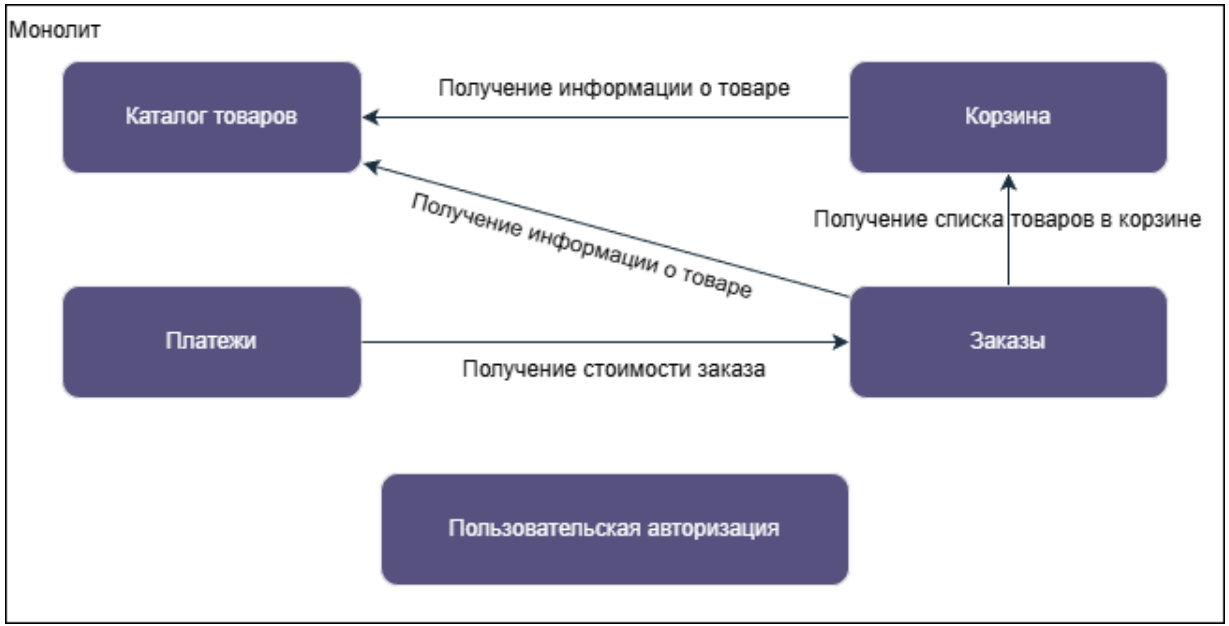
\+ если есть вопросы

– если вопросов нет

Кейс 1

Вводные

Компания «ShopNow» использует монолитную систему, написанную 5 лет назад на Java Spring. Ниже показаны модули, реализованные в монолите, и взаимодействия между ними :



Вопросы

Код стало сложно поддерживать: малейшие изменения в каталоге требуют пересборки всего приложения. Решено перейти на микросервисы. Команда обсуждает стратегию миграции. Архитектор предлагает: «Давайте сразу вынесем заказы и платежи в отдельные сервисы и отключим их в монолите». Однако инженеры опасаются просто "вырезать" функциональность —система слишком взаимосвязана.

1. Почему «вырезание» модулей из монолита может быть рискованным?
2. Как можно уменьшить риски при миграции без остановки системы? Что позволит постепенно переносить функциональность без полного отказа от монолита?
3. Какие паттерны можно использовать для вынесения функционала из монолита в отдельный микросервис?
4. Какую архитектурную схему нужно реализовать на уровне API Gateway?
5. Какие сервисы логично вынести первыми? Почему?

Возможные ответы:

1. *Почему «вырезание» модулей из монолита может быть рискованным?*
   Прямое вырезание нарушает зависимости, может привести к неработающим вызовам и сбоям в продакшене.
2. *Как можно уменьшить риски при миграции без остановки системы?Что позволит постепенно переносить функциональность без полного отказа от монолита?*
   Использовать постепенный перенос с параллельным существованием старого и нового кода. Реализовать паттерн Strangler, который позволяет "обвивать" монолит и постепенно заменять его модули.
3. *Какие паттерны можно использовать для вынесения функционала из монолита в отдельный микросервис?*
   Strangler, API Gateway, Branch y abstraction
4. *Какую схему взаимодействия нужно реализовать на уровне API Gateway?*
   На уровне API Gateway—маршрутизация запросов: часть идёт в монолит, часть —в новые микросервисы.
5. *Какие сервисы логично вынести первыми? Почему?*
   Начинать с наименее связанных модулей (например, авторизация, платежи), чтобы минимизировать риски.

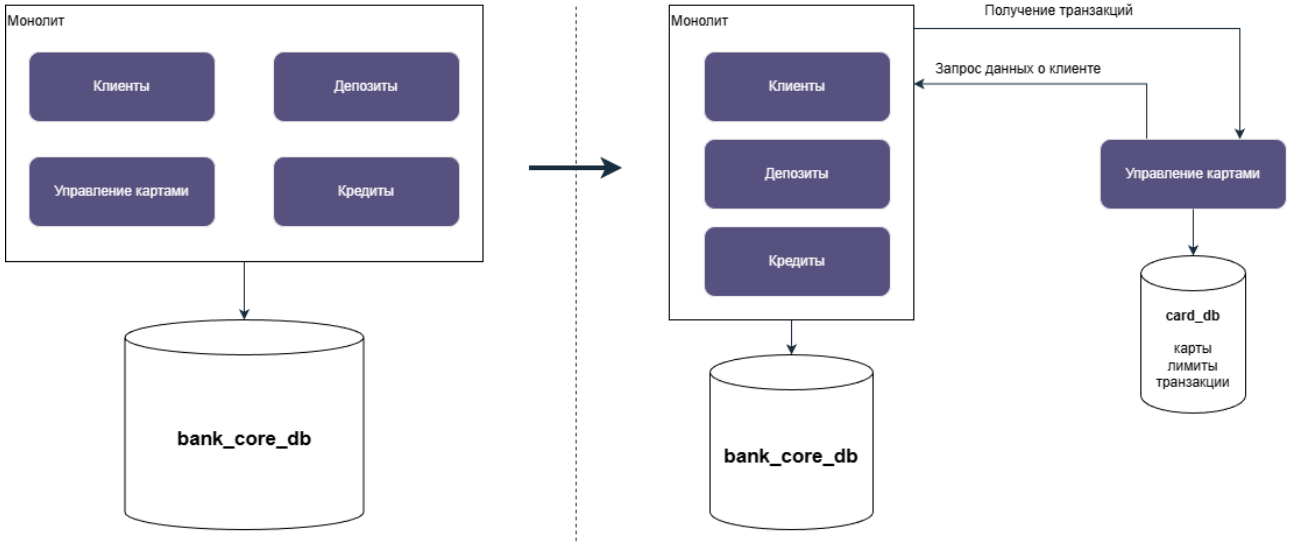
Возможное решение:

- Использовать паттерн Strangler: обернуть монолит API Gateway'ем, который будет проксировать запросы либо в монолит, либо в новые микросервисы. Новые эндпоинты реализуются в отдельных сервисах, старые постепенно вымирают.
- Начинать с сервисов, где меньше связей (например, авторизация и платежи).
- Можно первым этапом обернуть монолит API Gateway'ем и вынести аутентификацию в отельный сервис и перенести аутентификацию на уровень API Gateway.
- Преимущества: минимизация рисков, возможность постепенного тестирования и переноса нагрузки.
- Риски: сложность синхронизации данных и маршрутизации, возможные дубли в логике на переходном этапе, нестабильная работа на этапе миграции, замедление разработки на этапе миграции.

Кейс 2

Вводные

Компания «FastBank» - классический банк, который предлагает клиентам широкую линейку продуктов: депозиты, кредиты, банковские карты и т.д. Компания активно развивает карточное направление, имеет широкую сеть банкоматов и платежных терминалов. Как следствие нагрузка на монолит неравномерна, большая часть запросов приходится на создание карточных транзакций. Поэтому было решено выделить микросервис «Управление картами» из монолита.



Вопросы

Проблема: сервис «Управление картами» должен быстро масштабироваться и обновляться независимо от остальных. Но если отделить его, нарушится множество SQL-запросов в других модулях, которые напрямую читают данные по банковским картам (например, получение карточных транзакций для формирования общей выписки). При этом, сервису Карты также требуется получать данные из монолита (например, данные о клиенте при выпуске новой карты).

1. Какие риски возникают, если микросервис и монолит продолжают использовать одну базу данных?
2. Какие проблемы могут возникнуть при миграции на отдельную БД?
3. Какие существуют варианты работы с данными при миграции? Какой вариант лучше подходит для постепенного выделения сервиса «Управление картами»?
4. Как решать проблему доступа к данным между сервисом карт и монолитом? Какие существуют варианты?

Возможные ответы:

1. *Какие риски возникают, если микросервис и монолит продолжают использовать одну базу данных?*
   Нарушается независимость сервисов, изменения в схеме могут ломать монолит или микросервис; проблемы с версионированием и масштабированием.
2. *Какие проблемы могут возникнуть при миграции на отдельную БД?*
   Прямые SQL-запросы к перенесенным данным из монолита, общие таблицы, которые используются одновременно и монолитом и микросервисом .
3. *Какие существуют варианты работы с данными при миграции? Какой вариант лучше подходит для постепенного выделения сервиса «Карты»?*
   Варианты: Shared Database, Database per Service, репликация или event-driven синхронизация. Для плавного перехода можно временно использовать Shared Database, затем перейти к Database per Service.
4. *Как решать проблему доступа к данным между сервисом карт и монолитом? Какие существуют варианты?*
   Решения: замена прямого получения данных на взаимодействие через Rest API, событийная интеграция (Kafka, RabbitMQ), репликация таблиц.

Возможное решение:

- На этапе миграции допустимо использовать Shared Database как временное решение, но с постепенным переходом к Database per Service.
- Возможные промежуточные варианты:
  - Создать прослойку API для доступа к данным карт (вместо прямого SQL из монолита).
  - Реализовать репликацию данных или событийную синхронизацию (например, через Kafka).
    - Для согласованности: применить event-driven подход, при котором сервис карт публикует события, а монолит подписывается.
    - Итог: подготовка к полному разъединению базы и переходу на независимые схемы.

Подведём итоги занятия

Цели вебинара

Проверка достижения целей

1. Применять паттерн Strangler
2. Применять паттерны работы с данными при рефакторинге

Рефлексия

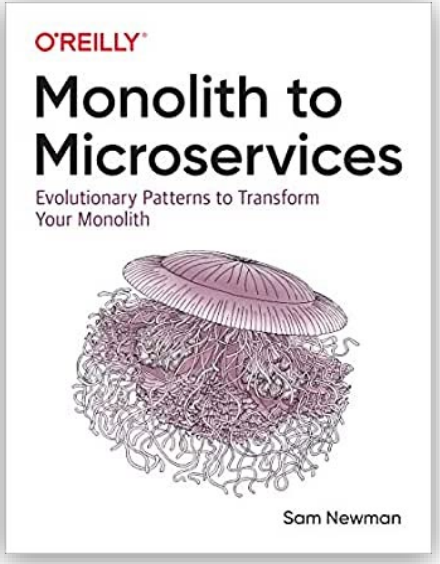


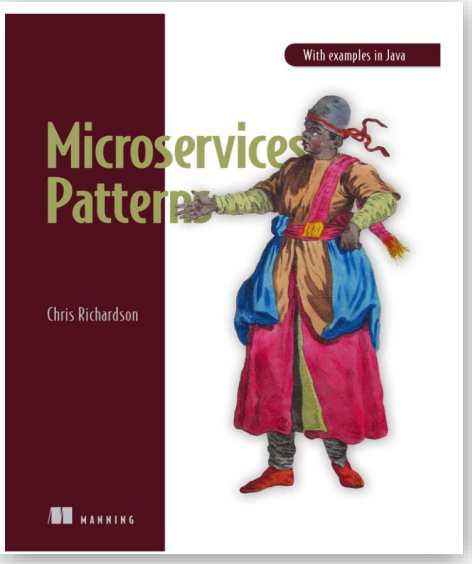
С какими впечатлениями уходите с вебинара?

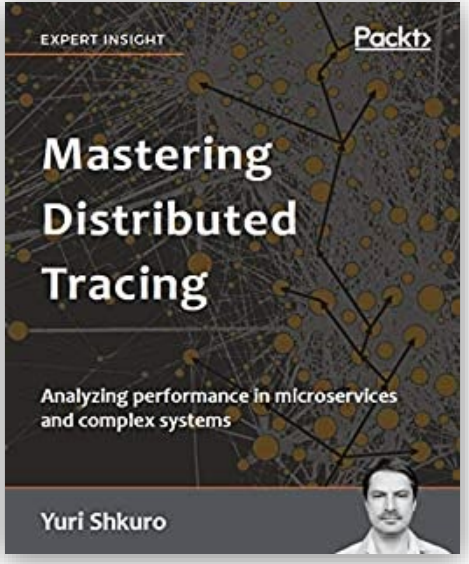


Как будете применять на практике то,
что узнали на вебинаре?

По этой теме вы можете изучить:







Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу ✏️❤️

Приглашаем на следующий вебинар: “Паттерны декомпозиции микросервисов // ДЗ ”

Ссылка на вебинар будет в ЛК за 15 минут

Материалы к занятию в ЛК — можно изучать

Обязательный материал обозначен красной лентой